

PilotFish HTTP Post Listener – Complete Reference (26R1.11 WAR)

Derived from decompiled eip.war.hs.26R1.11

August 4, 2026

At a glance (plain English)

Think of this as a **simple HTTP POST endpoint** on your PilotFish server. External systems POST a payload to a URL path you configure; each request starts an integration transaction. Optionally the listener waits for your route to finish and returns the response body over HTTP.

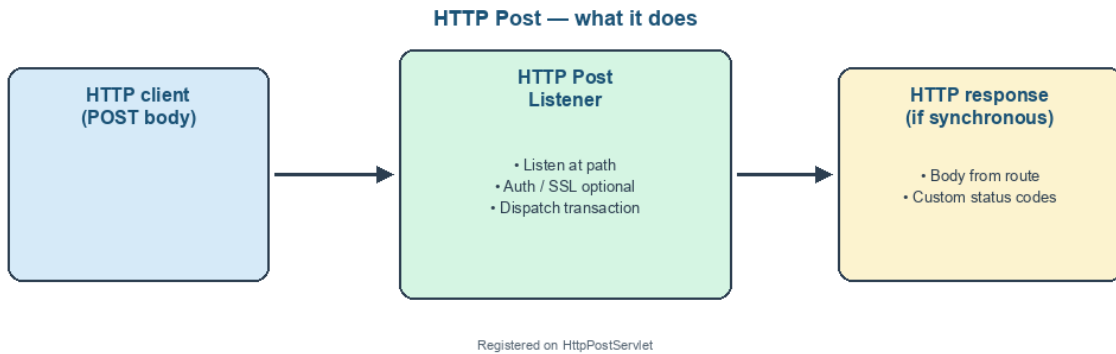


Figure 1: At a glance (plain English)

- **Listens** on a configurable HTTP path via `HttpPostServlet`.
- **Supports** synchronous responses, Basic auth, SSL requirement, and WSDL.
- **Maps** request metadata and headers to transaction attributes.
- **Validates** inbound XML against WSDL schema when enabled.

Purpose

Provenance	Value
WAR	eip.war.hs.26R1.11
Module JAR	xcs-modules-http-1.0.3.jar
Implementation class	com.pilotfish.eip.modules.http.HttpPostListener
Listener type string	HTTP Post
Help ID	console.listener.httppost

1. High-level behavior

1. **startListening**: normalizes Request path (leading /), registers additional resource paths, `HttpPostServlet.addPostListener`.
2. **postReceived(event)**: Require SSL check -> Basic auth -> read body -> inner processing.
3. **Inner postReceived(data, request)**: optional WSDL validation -> build attributes -> async `notifyListeners` or sync `dispatchForSynchronousResponseTransactionData`.
4. **Sync response**: route stream returned; optional `%CODE:NNN%` prefix for custom status.
5. **setResponseHeaders**: applies configured HTTP Headers table + **StoredHeaders** from route.

2. Configuration reference

Label	Tag	Default	Tab	Notes
Request path	<code>RequestPath</code>	""	Basic	Required; leading / added if missing
Timeout	<code>Timeout</code>	600	Basic	Sync wait seconds
Synchronous	<code>Synchronous</code>	false	Advanced	Shows HTTP Headers when true
Require SSL	<code>RequireSSL</code>	false	Advanced	Non-secure -> 405
Timeout handling	<code>TimeoutHandling</code>	<code>Selectionism</code> message	Advanced	See choices below
HTTP Headers	<code>HTTPHeaders</code>	Content-Type pair	Advanced	Sync response headers
WSDL file	<code>WsdFile</code>	-	WSDL	Served on <code>?wsdl</code>
WSDL Content Type	<code>WsdContentType</code>	<code>application/wsdl+xml</code>	WSDL	
Validate At The Listener Level	<code>ValidateAtListenerLevel</code>	false	WSDL	RPC or document style validation
Use Basic Authentication	<code>USE_BASIC_AUTH</code>	false	Authentication	
Authentication File	<code>AUTH_FILE</code>	-	Authentication	Java properties user=password

Label	Tag	Default	Tab	Notes
Authentication Realm	REALM	PilotFish	Authentication	
Additional Resources	AdditionalResources		Resources	URL fragment, file, content-type

Timeout handling: Send timeout message (default body), Return original request, Throw listener exception.

Shared HTTP Post servlet behavior

All servlet-based POST listeners register with `HttpPostServlet.addPostListener`. Duplicate path -> error logged, listener does not start.

Synchronous response protocol (%CODE): When route sets `com.pilotfish.HTTPResponseCode`, response stream may be prefixed with `%CODE:NNN%` + body bytes; outer servlet layer strips prefix and sets HTTP status.

Async acknowledgment: Many paths return UTF-8 `pf:default` or `POST Received` when not waiting for route output.

WSDL: Listeners implementing `ExpandedWSDLPostListener` serve configured WSDL when request ends with `?wsdl`.

3. WSDL validation algorithm

When **Validate At Listener Level** true and WSDL loaded in `initializeListener`:

- **RPC style:** root element QName must match operation input message; required parts must exist as child elements.
- **Document style:** validates against embedded XSD from WSDL via `Validator`.
- Validation failure -> returns **null** stream (error logged); request does not dispatch.

4. Transaction attributes

Registered: `com.pilotfish.RequestPath`, `ServerName`, `ServerPort`, `RemoteAddr`, `RemoteHost`, `RequestURL`, `Protocol`, `Locale`.

Runtime: `com.pilotfish.{Header}`, `com.pilotfish.Content-Type`, `com.pilotfish.http.parameter.{name}`, `com.pilotfish.Synchronous`, `com.pilotfish.eip.modules.http.StoredHeaders`.

Sync response: `com.pilotfish.HTTPResponseCode`, `StoredHeaders` map.

5. Synchronous %CODE protocol

Outer wrapper detects body starting with %CODE (length 10): extracts 3-digit code at positions 6-9, sets servlet status, returns remainder as body. Inner sync path builds this when `HTTPResponseCode` attribute set.

6. Known quirks

1. **Async** returns `pf:default` bytes (not business payload).
 2. **Auth password split** uses `[:\s+]` regex – unusual delimiters accepted.
 3. **Require SSL** uses HTTP 405 (not 403).
 4. **Validation null return** may yield empty/error HTTP response depending on servlet layer.
 5. **setImmediateSave(true)** on descriptor.
-

7. Worked example

Request path `/ingress/order`, Synchronous true, Timeout 120, HTTP Headers Content-Type `application/json`.

1. Client POSTs JSON body to `https://host/eip/http-post/ingress/order`.
2. Attributes include headers, RemoteAddr, body stream.
3. Route sets `HTTPResponseCode=201` and JSON body.
4. Client receives 201 with route output.

Inherited AbstractListener options

From `AbstractListener.getBaseConfiguration()` (Advanced, Throttling, Inactivity, Scheduling tabs):

Label	Tag	Default
Initialize on trigger only	<code>IS_TRIGGERABLE_LISTENER</code>	<code>true</code>
Allow command-line invocation	<code>CLIAllowed</code>	<code>false</code>
Restart on listening error	<code>RESTART_ON_ERROR_TAG</code>	<code>false</code>
FIFO Queue Name	<code>FIFOQueueName</code>	<code>" "</code>
FIFO Queue Delay	<code>FIFOQueueDelay</code>	<code>500</code>
Throttling Mode	<code>ThrottlingMode</code>	<code>None</code>
Throttling Mechanism	<code>ThrottlingMechanism</code>	<code>Blocking</code>
Max Concurrent Messages	<code>ThrottlingConcurrentMessages</code>	<code>50</code>
Timed Emission Interval	<code>ThrottlingTimedInterval</code>	<code>1</code>
Synchronous Timeout Interval	<code>ThrottlingSynchronousTimeout</code>	<code>500</code>
Enable Inactivity Monitor	<code>CheckInactivity</code>	<code>false</code>
Min. Transactions to Expect	<code>InactivityTransactionCount</code>	<code>1</code>
Monitoring Interval	<code>InactivityPolling</code>	<code>500</code>

Label	Tag	Default
Excluded Days / Dates	<code>InactivityExcludedDays</code> , <code>InactivityExcludedDates</code>	
Start / End schedule	Scheduler items	–

Scheduling uses Quartz cron when configured; polling listeners also honor engage/disengage lifecycle.

8. Source index

Topic	Path
HttpPostListener	<code>com/pilotfish/eip/modules/http/HttpPostListener.java</code>
HttpPostServlet	<code>com/pilotfish/eip/modules/http/HttpPostServlet.java</code>
WSDLUtils	<code>com/pilotfish/eip/modules/http/wsdl/WSDLUtils.java</code>

Deep-dive documentation from WAR decompile – 26R1.11 – August 2026.